Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Разработка программного обеспечения для мобильных платформ

ОТЧЕТ

по лабораторной работе №3

Каталог видеоигр и игровых студий с рейтингами

Выполнил
Студент гр. 351002 Михальков А. Д.

Проверил Петровская В. В.

Минск 2026

СОДЕРЖАНИЕ

1 Введение 3
1.1 Функциональные изменения 3
1.1.1 Продвинутые инструменты работы с данными 3
1.1.2 Система уведомлений 4
1.1.3 Интеграция с удаленной базой данных (Firebase Firestore) 5
1.1.4 Облачное хранилище (ImageKit) 5
1.2 Теоретическая основа 6
1.3 Технологии 7
2 Скриншоты работы приложения 8
2.1 Демонстрация нечеткого поиска и сортировки 8
2.2 Настройка и получение уведомлений 8
2.3 Вкладка «История» 10
2.4 Загрузка изображений из галереи 10
2.5 Верификация данных в Firebase Firestore 11
3 Заключение 13

1 Введение

Цель работы: Расширение функциональных возможностей мобильного приложения путем внедрения механизмов облачной синхронизации данных, интеграции удаленного хранилища медиаконтента, реализации интеллектуальных алгоритмов поиска и системы персонализированных уведомлений.

В рамках данной лабораторной работы решаются следующие задачи:

1. Облачная интеграция: Переход от изолированного локального хранилища к распределенной модели данных на базе Firebase Firestore, что обеспечивает сохранность информации и возможность синхронизации между различными устройствами пользователя.
2. Реализация системы загрузки и хранения пользовательских изображений в облачном сервисе ImageKit, что позволяет оптимизировать использование памяти устройства.
3. Обработка данных: Внедрение алгоритмов нечеткого поиска, позволяющих эффективно находить информацию при наличии опечаток или неполных входных данных, а также разработка расширенных инструментов сортировки и фильтрации контента.
4. Система уведомлений: Проектирование и реализация механизма локальных уведомлений по расписанию с сохранением персональной истории событий в базе данных.

1.1 Функциональные изменения

1.1.1 Продвинутые инструменты работы с данными

В рамках модернизации приложения был переработан механизм взаимодействия пользователя с контентом. Основной упор сделан на повышение гибкости поиска и удобство навигации по большим массивам данных. Реализованный функционал включает три ключевых направления.

Алгоритм нечеткого поиска: для устранения ограничений стандартного поиска по точному совпадению подстроки в проект была интегрирована библиотека Fuse.js. Данное решение реализует поиск на основе алгоритмов приближенного сравнения (в частности, расстояния Левенштейна).

Инструментарий сортировки: для систематизации отображаемого списка во ViewModel была внедрена логика динамической сортировки. Пользователю предоставлена возможность мгновенного изменения порядка записей по трем параметрам:

1. По названию: использование метода localeCompare обеспечивает корректную обработку кириллических и латинских символов с учетом регистра;
2. По рейтингу: сортировка по убыванию, позволяющая быстро находить наиболее высокооцененные игры;
3. По дате: актуально для данных, получаемых из внешнего Web API. Система преобразует строковые значения дат в объекты типа Date для корректного ранжирования от новых релизов к старым.

Динамическая фильтрация: механизм фильтрации был расширен и теперь работает на двух уровнях:

1. Глобальный уровень: разделение контента по вкладкам;
2. Категорийный уровень: на вкладке «Global» реализована горизонтальная лента фильтров по жанрам. Выбор категории инициирует параметризованный запрос к серверу RAWG API, возвращая пользователю точечную выборку данных.

1.1.2 Система уведомлений

Для улучшения взаимодействия с пользователем была внедрена система локальных уведомлений. В отличие от стандартных систем оповещения, реализованное решение сочетает в себе механизмы планирования событий и ведение персонального архива уведомлений.

1. Реализация базируется на использовании библиотеки expo-notifications, которая позволяет взаимодействовать с системным менеджером уведомлений. Система включает в себя логику предварительной проверки и динамического запроса разрешений у операционной системы.
2. Настроено два вида событий:
   - Интервальные: тестовое уведомление, срабатывающее через фиксированный промежуток времени после активации функции для мгновенной обратной связи;
   - Календарные: ежедневные напоминания, запланированные на определённое время.
3. Ключевой особенностью системы является наличие «Центра уведомлений» (вкладка «История»). В локальной базе данных SQLite создана специализированная таблица notification_history, содержащая заголовок, текст сообщения и точную метку времени получения.
4. Логика хранения уведомлений интегрирована с системой авторизации. Каждая запись в истории снабжается идентификатором владельца owner_id. Это обеспечивает разграничение данных: при смене аккаунта список уведомлений полностью обновляется, отображая только те события, которые были адресованы текущему

пользователю. Для «Гостевого режима» предусмотрена отдельная временная история.

1.1.3 Интеграция с удаленной базой данных (Firebase Firestore)

Для реализации облачной синхронизации была выбрана платформа Firebase Firestore, представляющая собой документоориентированную NoSQL базу данных реального времени.

1. Реализация CRUD:

Взаимодействие с удаленным сервером полностью интегрировано в бизнес-логику приложения:

- Создание: при добавлении новой игры данные асинхронно передаются в коллекцию games облачного хранилища. Операция возвращает уникальный идентификатор документа, который мгновенно фиксируется в локальной базе;
- Чтение: реализована логика запросов с фильтрацией на стороне сервера, позволяющая приложению запрашивать только те данные, которые принадлежат текущему авторизованному пользователю;
- Обновление: редактирование информации об игре инициирует транзакцию обновления конкретного документа в Firebase по его уникальному ID, обеспечивая актуальность данных на всех устройствах пользователя;
- Удаление: при удалении записи из локального интерфейса срабатывает триггер на удаление соответствующего документа из Firestore, что гарантирует отсутствие избыточных данных в облаке.

2. Доступ к облачному хранилищу осуществляется через сконфигурированный Firebase SDK. Для обеспечения целостности данных в структуре каждого документа предусмотрено хранение метаданных, таких как временные метки создания (createdAt) и обновления (updatedAt), а также привязка к owner_id пользователя. Это позволяет организовать использование сервиса, где каждый клиент имеет доступ только к своему сегменту данных.

1.1.4 Облачное хранилище (ImageKit)

Для перехода от использования внешних ссылок к собственному хранилищу медиафайлов, в приложение была интегрирована система управления изображениями ImageKit.

1. Процесс загрузки и обработки изображений:

Техническая реализация взаимодействия с облачным хранилищем построена на использовании программного интерфейса ImageKit Upload API. С помощью модуля expo-image-picker реализован доступ к файловой системе устройства, позволяющий пользователю выбирать изображения непосредственно из памяти телефона. Выбранный файл передается на сервер через POST-запрос с использованием объекта FormData. При загрузке сервис автоматически оптимизирует изображения, предоставляя приложению прямую CDN-ссылку (URL), адаптированную для быстрой доставки контента на мобильные устройства.

2. Управление жизненным циклом файлов:

Одной из ключевых особенностей реализации является хранение не только публичной ссылки на изображение, но и его уникального идентификатора в системе облака – fileId. При удалении записи об игре или при замене фотографии в режиме редактирования, система инициирует DELETE-запрос к API ImageKit. Таким образом, старый файл физически удаляется с сервера.

Сервис инкапсулирован в отдельный программный модуль ImageService.js, который включает обработку сетевых исключений. В случае недоступности облака или ошибки загрузки, система использует локальные пути к файлам или значения по умолчанию.

1.2 Теоретическая основа

Теоретический базис разработки охватывает принципы работы с распределенными данными, алгоритмы интеллектуального поиска и механизмы фонового взаимодействия мобильного приложения с операционной системой.

В основе реализованного алгоритма нечетного поиска лежит метрика расстояния Левенштейна. Данный алгоритм вычисляет минимальное количество операций вставки, удаления и замены одного символа, необходимых для превращения одной строки в другую. Применение данного алгоритма позволяет приложению возвращать результаты даже при наличии в поисковом запросе опечаток, пропущенных символов или неверного регистра.

В проекте реализована гибридная архитектура данных, сочетающая преимущества двух типов СУБД.

Реляционная модель (SQLite) обеспечивает строго структурированное хранение данных на устройстве, транзакционную целостность и высокую скорость локальных выборок.

Документоориентированная модель (NoSQL / Cloud Firestore) хранит данные в виде коллекций и документов. Такая модель обладает высокой горизонтальной масштабируемостью.

Использование ImageKit базируется на теории Digital Asset Management. Вместо хранения бинарных данных изображений в базе данных, используется хранение ссылок на оптимизированные узлы сети доставки контента (CDN). Это обеспечивает минимальную задержку при загрузке графики и снимает нагрузку с внутренней памяти устройства.

1.3 Технологии

Для реализации расширенного функционала приложения в рамках работы были интегрированы следующие программные средства и библиотеки:

1. Firebase SDK (Firestore): облачная платформа от Google, использованная для реализации удаленной базы данных NoSQL;
2. ImageKit API: сервис управления данными, интегрированный через REST API. Позволяет организовать загрузку изображений в облако, их автоматическую оптимизацию через CDN и программное удаление файлов с сервера;
3. Fuse.js: легковесная библиотека для реализации нечеткого поиска на стороне клиента. Позволяет выполнять поиск по нескольким полям данных одновременно с использованием алгоритмов приближенного сопоставления строк;
4. expo-notifications: системный модуль для работы с уведомлениями в экосистеме Expo. Предоставляет доступ к API операционных систем для планирования локальных уведомлений, управления разрешениями и настройки каналов оповещений.
5. expo-image-picker: Библиотека для взаимодействия с мультимедийными ресурсами устройства. Использована для реализации интерфейса выбора изображений из нативной галереи телефона.

2 Скриншоты работы приложения

2.1 Демонстрация нечеткого поиска и сортировки

На Рисунке 1 представлена работа алгоритма нечеткого поиска на базе библиотеки Fuse.js. Система успешно идентифицирует целевой объект даже при наличии опечаток в поисковом запросе, что подтверждает корректную настройку коэффициента чувствительности. Также продемонстрирована работа инструментов сортировки, позволяющих динамически упорядочивать записи по рейтингу и алфавиту.

Рисунок 1. Работа алгоритма нечеткого поиска и инструментов сортировки

2.2 Настройка и получение уведомлений

Рисунки 2, 3 демонстрируют процесс активации системы оповещений в настройках профиля и момент получения системного Push-уведомления. Уведомление содержит заголовок и текст, сформированные согласно текущим языковым настройкам интерфейса.

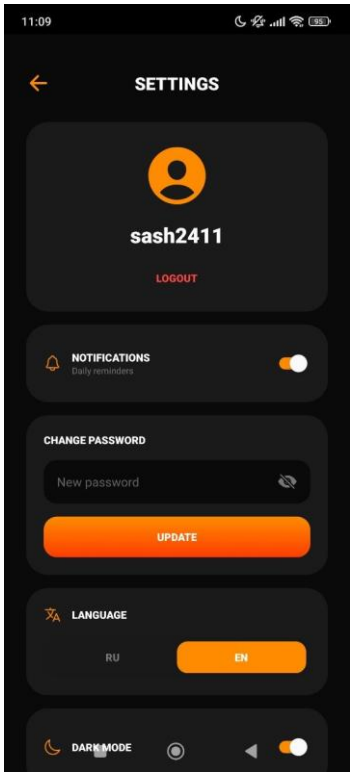

Рисунок 2. Активация системного уведомления

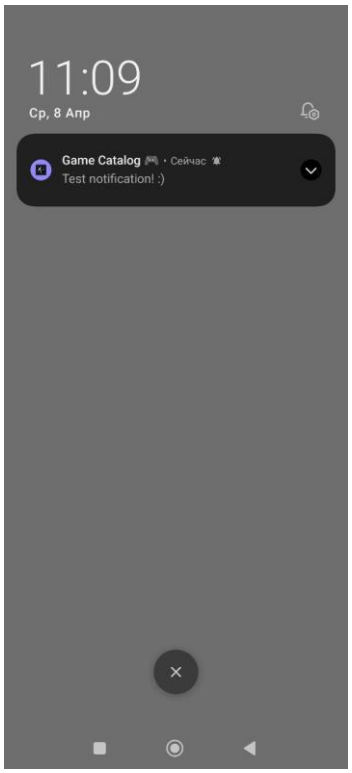

Рисунок 3. Получение системного уведомления

2.3 Вкладка «История»

На Рисунке 4 отображен интерфейс «Центра уведомлений». Каждое системное событие автоматически фиксируется в локальной таблице базы данных SQLite с привязкой к ID текущего пользователя. Записи содержат метаданные о времени получения и тексте сообщения.

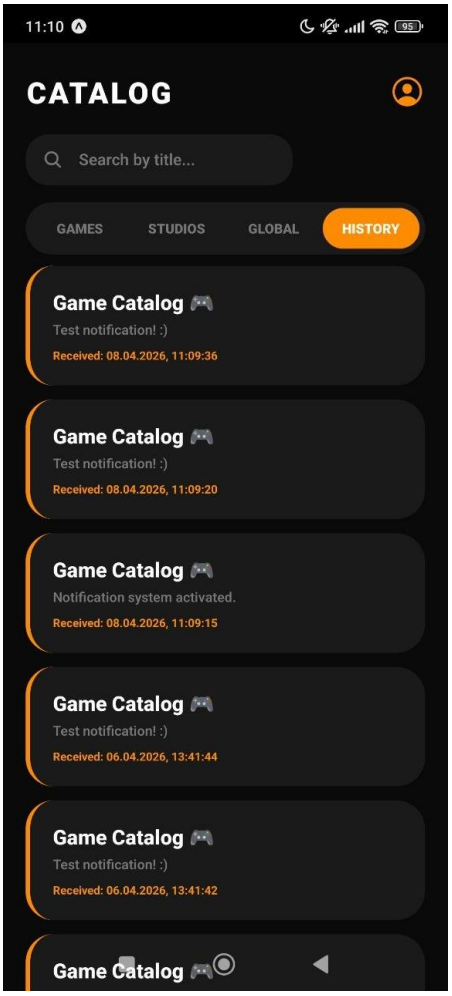

Рисунок 4. Интерфейс истории персональных уведомлений

2.4 Загрузка изображений из галереи

Процесс выбора медиаконтента через модуль expo-image-picker представлен на Рисунке 5. После выбора изображения из нативной галереи система формирует временное превью и инициирует асинхронную загрузку файла в облачное хранилище ImageKit с последующим получением оптимизированной CDN-ссылки.

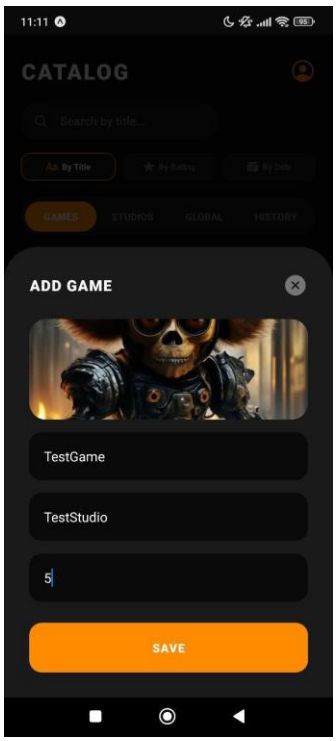

Рисунок 5. Выбор изображения из галереи и подготовка к загрузке в облако

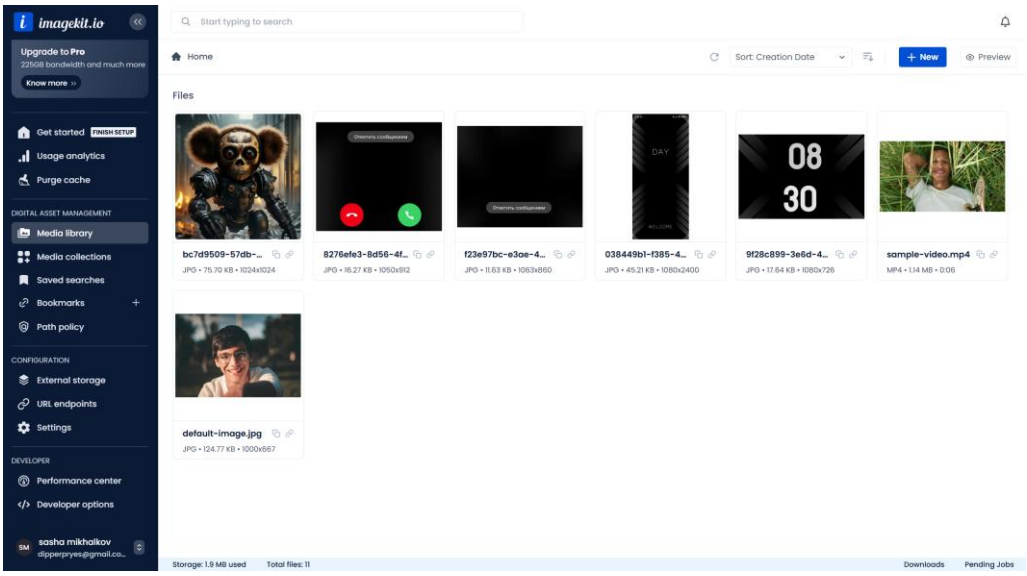

Рисунок 6. Изображение в облаке

2.5 Верификация данных в Firebase Firestore

На Рисунке 7 приведено подтверждение успешной синхронизации данных с удаленным сервером. В консоли управления Firebase Firestore зафиксированы документы коллекции games, структура которых полностью соответствует локальным моделям данных, включая идентификаторы владельцев и ссылки на облачные изображения.

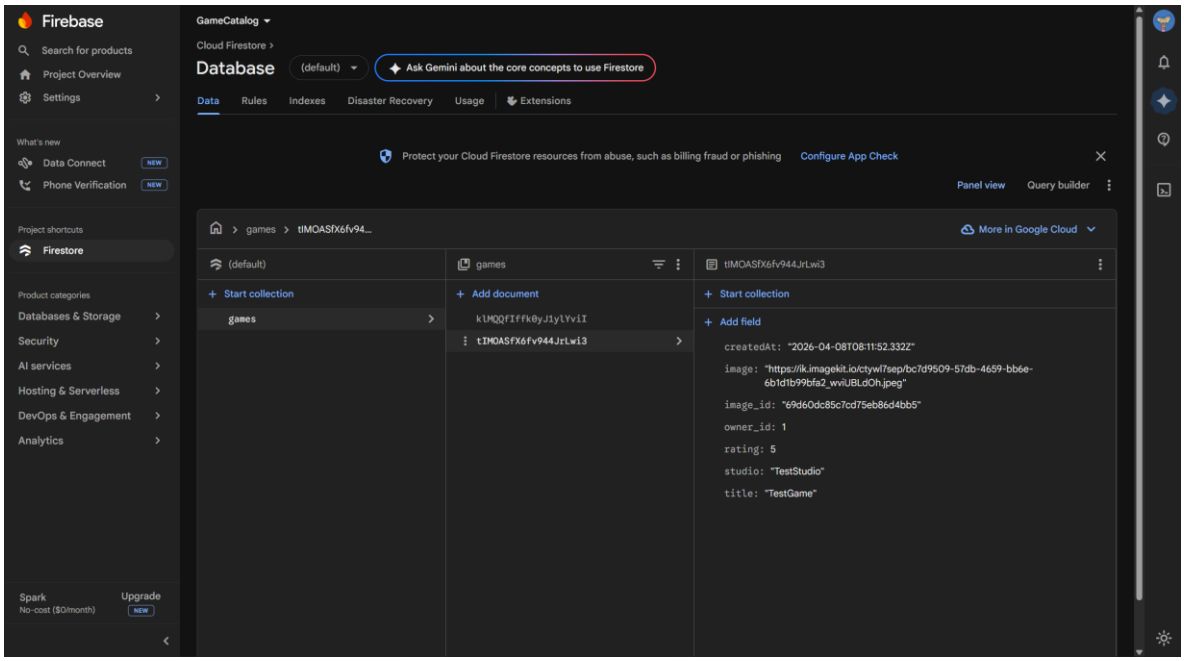

Рисунок 7. Верификация синхронизации данных в консоли Firebase Firestore

3 Заключение

В ходе выполнения работы был разработан полноценный клиент-серверный сервис.

Основной упор в работе был сделан на внедрение технологий распределенного хранения данных и облачной синхронизации, что позволило обеспечить сохранность пользовательского контента независимо от состояния устройства.

Интеграция с платформой Firebase Firestore обеспечила надежную связь между локальной базой данных SQLite и удаленным NoSQL-хранилищем, позволив реализовать синхронный цикл управления данными в режиме реального времени.

Параллельно с этим была внедрена система управления медиаконтентом на базе сервиса ImageKit, которая позволила не только организовать загрузку и оптимизированное хранение пользовательских изображений в облаке, но и реализовать алгоритм автоматической очистки серверного пространства при удалении или изменении записей.

Значительное внимание было уделено совершенствованию пользовательского опыта через инструменты интеллектуальной обработки информации.

Использование библиотеки Fuse.js позволило внедрить алгоритмы нечеткого поиска, устойчивого к ошибкам ввода, а расширенные механизмы сортировки существенно повысили удобство навигации по каталогу.

Все программные решения были успешно вписаны в существующую архитектуру MVVM, что подтвердило масштабируемость и надежность выбранного подхода.

Ссылка на репозиторий с итоговым кодом:
https://github.com/zuluw/game-catalog